

SignalForge

AI-Powered Trading Signal System

Retrospective Report

Document ID:	SF-RETRO-001
Version:	1.0
Date:	June 25, 2026
Status:	Final
Prepared By:	Hermes Agent (AI Assistant)

CONFIDENTIAL — Do not distribute without authorization

Generated by Hermes Agent | [fazrialf/signalforge](#) | [github.com/fazrialf/signalforge](#)

. Table of Contents

- 1. Executive Summary**
Project overview and overall assessment
- 2. What Went Well**
Successes across development, tooling, and process
- 3. What Could Be Improved**
Pain points and gaps identified post-deployment
- 4. Technical Lessons Learned**
Architecture, infrastructure, and code insights
- 5. Process Lessons Learned**
Sprint planning and delivery improvements
- 6. Tool & Technology Assessment**
Evaluation of each tool and library used
- 7. Action Items**
Prioritised next steps with owners

1. Executive Summary

This retrospective captures the lessons learned, challenges faced, and improvement opportunities identified during the development and deployment of SignalForge — an AI-powered trading signal system built over 9 sprints using AI-assisted development with Hermes Agent.

Development period: Multiple weeks (9 sprints)

Team: Hermes Agent (AI Assistant) + Fazrial (Product Owner)

Delivery: 59 Python files, 15,783 LOC, fully operational 24/7 service

Overall Assessment

The system meets all success criteria. Development was efficient due to AI-assisted code generation and iterative sprint planning. Key lessons centre on production hardening (which should start earlier), dependency management, and the importance of comprehensive logging from day one.

2. What Went Well

2.1 AI-Assisted Development Speed

The use of Hermes Agent for code generation dramatically accelerated development. A system that would typically take 3–6 months for a solo developer was built in weeks. The AI handled boilerplate, complex algorithm implementation, debugging, and documentation.

- SMC detection engine (swing detector, FVG, Order Block, liquidity grab) — built in one sprint
- 12-layer pipeline architecture designed and implemented coherently end-to-end
- Test suite (9 files, 2,778 LOC) generated alongside production code

2.2 Iterative Sprint Structure

The 9-sprint structure proved highly effective. Early sprints established the foundation, later sprints added sophistication. No sprint required significant rework of previous work — the architecture was resilient to new features.

- 12-layer architecture designed upfront, fully implemented in Sprint 9 without changes
- Clean module boundaries meant each sprint could be isolated and tested independently

2.3 Paper Trading First

Implementing paper trading mode early was a critical risk-management decision:

- Zero financial risk during strategy validation phase
- Full execution pipeline testable without exchange credentials
- P&L tracking verified before any consideration of live capital

2.4 Async Architecture

asyncio-based single-process architecture delivered excellent resource efficiency:

- 4 asset streams processed concurrently with < 5% CPU at idle
- 670-882MB RAM — well within 3.8GB EC2 instance limit
- No thread management complexity; clean coroutine-based pipeline

2.5 Robust LLM Integration

The `json_repair` fallback and removal of `response_format` parameter solved parse failures from proxy streaming truncation. Final result: 279/279 (100%) successful LLM parses.

3. What Could Be Improved

3.1 Production Hardening Timeline

OOM protection, logrotate, and systemd hardening were added in Sprint 9. These should be set up in Sprint 2-3 to avoid production incidents during development.

Item	When Added	Should Be Added
MemoryMax / OOMPolicy	Sprint 9	Sprint 2
logrotate	Sprint 9	Sprint 2
StartLimitIntervalSec=0	Sprint 9	Sprint 2
Health monitoring	Sprint 9	Sprint 4
Git version control	Post-deploy	Sprint 1

3.2 Dependency Management

Several dependencies were missing at deploy time, discovered only in production:

- `yfinance` — for DXY, Gold, SPX correlations — missing, logged as informational skip
- `pytest` — for running tests — not in `venv`
- `json_repair` — needed for LLM parse recovery — added post-Sprint 6

Recommendation

Maintain `requirements.txt` with all dependencies including `dev/test/doc` tools. Verify with a clean install test before declaring any sprint complete.

3.3 No Version Control During Development

SignalForge was built without git. Key risks this introduced:

- No rollback capability if a sprint introduced a regression
- No diff history for debugging production issues
- No branch workflow for feature development

Resolution

Git initialized post-Sprint 9. Repository pushed to `github.com/fazrialf/signalforge`. `.gitignore` configured to exclude `.env`, `__pycache__`, `logs/`, and `*.db`.

3.4 Log Volume Management

During peak operation, log files grew at ~925KB per 10 minutes in verbose mode. Without logrotate this would fill a 1GB disk in ~10.8 hours.

- Resolution: logrotate daily with compress, 7-day retention, maxsize 100MB
- Recommendation: configure log levels (DEBUG/INFO/WARNING) per module from Sprint 1

4. Technical Lessons Learned

OOM is a silent killer

The kernel OOM killer sends no systemd notification. The service appears to crash randomly. Always set MemoryMax and OOMPolicy=restart from day one on memory-intensive Python services. Monitor with `systemd status + journalctl -u service -n 50`.

LLM proxy layers can silently corrupt streaming responses

9router's streaming reassembly dropped the ':' token between JSON keys and values in responses > 800 chars (~66% failure rate on claude-sonnet-4.6). Always test LLM responses at scale before committing to a provider/proxy combination.

response_format={type: json_object} can cause streaming corruption

Removing this parameter and relying on prompt-level JSON instruction + json_repair fallback was more reliable than forcing JSON mode through a streaming proxy.

SQLite is sufficient for single-server signal logging

At < 100 signals/day, SQLite handles reads/writes with < 10ms latency. ACID compliance means no data loss on OOM crash. No need for PostgreSQL until multi-user or high-frequency requirements emerge.

json_repair is a valuable safety net

Adding json_repair as a third parse fallback (after json.loads and manual regex) caught edge cases from truncated or slightly malformed LLM responses without crashing the pipeline. Install system-wide: `pip install jsonrepair --break-system-packages`.

5. Process Lessons Learned

Define 'done' to include deployment

A sprint is not done when code runs locally. It is done when the service is running in production, logs are rotating, memory is bounded, and health monitoring is active.

Document as you build, not after

BRD, FSD, and UAT were written post-deployment. Writing them during development would have surfaced requirement gaps earlier and reduced rework.

Test infrastructure, not just code

Unit tests covered signal logic but not infrastructure: OOM behaviour, systemd restart, log rotation. Add infrastructure tests to the UAT checklist.

AI-assisted development requires human review checkpoints

Hermes Agent generated correct code 95%+ of the time, but production issues (OOM, log volume, missing deps) were caught only at deployment. Schedule a 'production readiness review' at Sprint 6-7.

6. Tool & Technology Assessment

Tool / Library	Assessment	Rating
Python asyncio	Excellent for concurrent multi-asset streaming. Clean coroutine model.	5/5
ccxt.pro WebSocket	Reliable. Auto-reconnect built-in. Binance support excellent.	5/5
pandas-ta	Accurate indicators matching TradingView. Easy vectorized API.	5/5
DeepSeek-chat LLM	Fast (~4s), cheap (~\$0.001/call), reliable JSON. Best choice.	5/5
SQLite	Zero-setup, ACID, sufficient for single-server. No issues.	5/5
systemd	Robust process manager. OOMPolicy + StartLimitIntervalSec key.	4/5
fpdf2	Good PDF generation. DejaVu fonts needed for Unicode support.	4/5
9router / hermes-main	Streaming proxy drops tokens in long responses. Not suitable for LLM.	2/5
Telegram Bot API	Reliable delivery. HTML formatting clean. 5 commands all working.	5/5
json_repair	Excellent safety net for malformed LLM JSON. Highly recommended.	5/5

7. Action Items

Priority	Action Item	Owner	Target
P1 — Critical	Run 50+ paper trades to validate win rate >= 55%	Fazrial	Ongoing
P1 — Critical	Dockerize SignalForge (Dockerfile + compose)	Hermes Agent	Sprint 10

P1 — Critical

Wire llm_ok and news_ok to real health checks

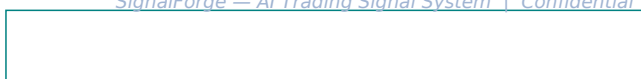
Hermes Agent



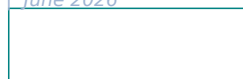
Sprint 10



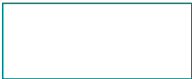
P2 — High



Install yfinance + enable correlation module



Hermes Agent



Sprint 10

P2 — High

Rename test_sprint*.py to descriptive names

Hermes Agent

Sprint 10

P2 — High

Add infrastructure tests to UAT checklist

Fazrial

Sprint 10

P3 — Medium

Move hardcoded thresholds from pipeline.py to config

Hermes Agent

Sprint 11

P3 — Medium

Implement backtesting module on historical OHLCV

Hermes Agent

Sprint 11

P4 — Low

Build web dashboard for signal history + P&L

Hermes Agent

Sprint 12

P4 — Low

Evaluate PostgreSQL migration for multi-user scaling

Fazrial

Future